

SEOUL — 3D ATLAS · v5.4

# 손안의 서울

Seoul in miniature



A MORE VIVID SEOUL  
IN YOUR HANDS

서진호  
OpenAI Codex  
Korea

(<https://www.facebook.com/groups/1100405985976972>)

# 오늘 할 이야기는?

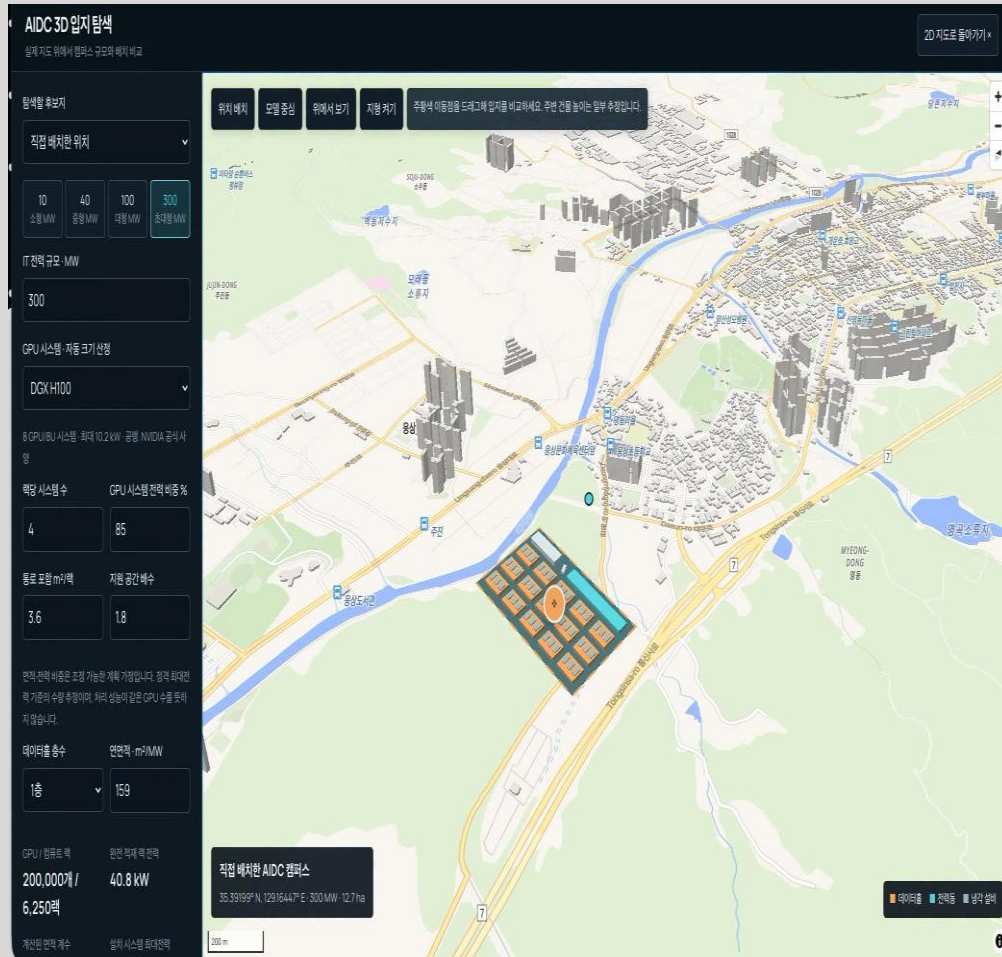
- 어떻게 시작했나?
- GPT-6 Astra에 맞는 Codex 프롬프트 작성법
- GPT-6 Astra Agents가 기존 에이전트와 다른 점
- GPT-6 Astra SDK/API – Codex Migration
- GPT-6 Astra는 AGI이다? 아니다!
- OpenAI GPT-6 Astra 생태계
- 결론
- Q&A

# 소셜 네트워크 반응

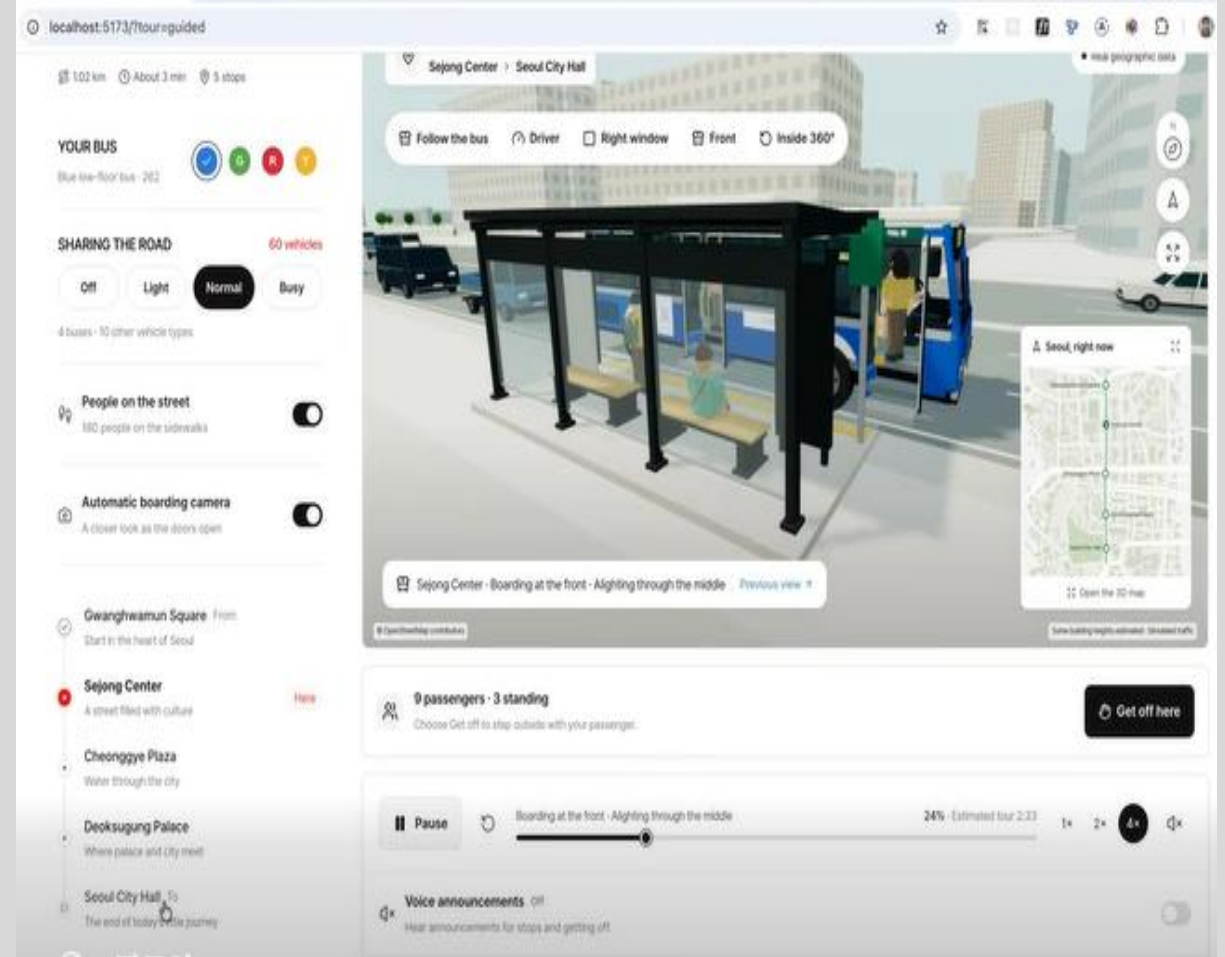


- ChatGPT 공식 계정 소개
- 트위터(X)
  - 33만 조회수 (335.4K)
  - 505 공유수
  - 2.100 북마크 (2.1K)
  - 댓글 97
- Facebook
  - 1만 조회수
  - 하트 수 204개
  - 공유 수 30개
- Thread
  - 하트 수 200개
  - 댓글 48개
  - 공유 22개

# 사양 사례



◦ 전종홍 박사 – AIDC 3D 시뮬레이터



◦ Anthony Kim – 서울지도 위를 달리는 버스



1. 어떻게 시작하  
게 되었나?

# 한 문장으로 시작한 서울 3D 지도

- 새 GPT 6 Astra 성능이 얼마나 되고, 어디까지 되는 가는 개인적인 호기심에 시작했다.
  - **Prompt 명령 : 서울 전체를 3D 지도처럼 만들어 달라.**
  - **(내 머리 속의) 내 아이디어**
    - 특정 동네나 몇 개의 랜드마크만 표현하는 수준이 아닌 서울 전체 자치구(25개)와 한강, 산, 도로, 약 26만 7천 개의 건물을 한 화면에서 둘러볼 수 있는 인터랙티브 3D 도시를 만들고 싶었다.
    - 사용자는 지도를 자유롭게 회전하고 확대하거나 이동할 수 있어야 한다.
    - 서울의 주요 지역과 랜드마크를 따라 자동으로 비행하고, 낮과 노을, 야경을 볼 수 있다.
      - 나중에 사용자의 피드백을 받아들여 비와 눈이 내리는 서울도 경험할 수 있도록 확장
  - **생성 결과 후 놀라운 점**
    - 서울 전체를 3D로 표현하는 것은 웹페이지 하나를 만드는 작업 통합 (속도와 성능)
  - 지도 데이터를 수집하고 좌표를 변환하며, 건물 윤곽을 3D 지오메트리로 바꾸고, 지형과 도로, 한강을 하나의 장면으로 결합함

# Astra가 만든 지도 결과의 놀라운 점은?

- Astra 가 서울 전체를 3D로 표현하고 웹페이지 하나를 만들어 통합시킴
- Astra 가 스스로 지도 데이터를 수집하고 좌표를 변환하며, 건물 윤곽을 3D 지오메트리로 바꾸고, 지형과 도로, 한강을 하나의 장면으로 결합함
- 무엇보다 수십만 개의 건물을 브라우저에서 렌더링하면서도 사용자가 불편하지 않을 정도의 성능을 유지함
- 프로젝트를 생성한 시점부터 최초 공개 버전을 배포하기까지 걸린 시간은 43분 38초 걸렸음.
  - 아이디어가 최초의 실행 가능한 웹서비스로 만들어지고 배포되기까지의 시간
  - 실제 지도를 사용하면서 문제를 발견했고, 기능과 표현을 계속 수정했음 (반복 작업)
  - 한 단계 더 나아가 3D 공간과 부동산 실거래 데이터를 연결한 서울 아파트 실거래 추가
  - 처음에는 서울을 '보는 지도'를 만들었다면, 두 번째 프로젝트에서는 서울을 '데이터로 탐색하는 인터페이스'로 확장함

# Astra가 서울 전체를 작은 세계로 만들다!

- Astra는 서울 25개 자치구와 약 26만 7천 개의 건물 모형을 KOSTAT 2013년 행정구역 경계와 JUSO 2015년 서울 행정구역 자료 기반으로 데이터를 가져와 포함시켰다.
- 건물은 OpenStreetMap에 등록된 위치와 윤곽을 단순화해 Astra가 만들었다.
  - 높이 정보가 없는 건물은 원본 데이터에 포함된 추정값이나 보정된 값을 사용했다.
  - 서울의 지형과 스카이라인을 한눈에 보기 쉽도록 지형과 건물 높이는 실제 비율보다 4배 강조했다.
- 이 서비스는 정밀한 측량이나 내비게이션을 위한 디지털 트윈이 아니다!
  - 공개 지도 데이터를 기반으로 서울의 구조와 분위기를 압축한 인터랙티브 3D 미니어처에 가깝다.
  - 처음부터 사진처럼 사실적인 도시를 복제하려 했던 것은 아니다. 내가 만들고 싶었던 것은 평면 지도에서 쉽게 느끼기 어려운 서울의 공간 구조였다.
  - 한강이 도시를 어떻게 나누는지, 북한산과 관악산이 서울을 어떻게 둘러싸고 있는지, 강남과 여의도, 광화문과 잠실이 어떤 관계로 배치돼 있는지를 하나의 입체적인 장면에서 보여주고 싶었다.

# 첫번째 결과는 완성품이 아니었다

- 초기 버전은 빠르게 만들어졌지만 모든 것이 처음부터 정확하지는 않았다.
  - 강남역과 삼성동 COEX가 하나의 지역처럼 표현됐고, 여의도의 비율 때문에 주변 한강과 샛강이 잘 드러나지 않았다.
  - 김포공항에서 서해 방향으로 이어지는 한강 하류도 중간에서 물길이 끊어진 것처럼 보였다.
- 나는 지도를 직접 사용하며 이런 문제를 하나씩 발견하고 개선시켰다.
  - Prompt 명령: 강남역과 COEX의 위치를 분리해 달라.
  - Prompt 명령: 여의도 주변으로 흐르는 한강과 샛강이 보이게 해 달라.
  - Prompt 명령: 김포공항에서 서해 쪽으로 이어지는 물길을 지도 끝까지 파란색으로 표시해 달라
- Astra는 이런 자연어 피드백을 데이터와 코드의 수정 사항으로 바꿨다.
  - 랜드마크 좌표와 카메라 이동 경로를 조정하고, 물길의 지오메트리와 색상을 수정한 뒤 새로운 버전으로 다시 배포했다.
  - 이 과정에서 내(인간)가 맡은 역할은 단순히 프롬프트를 입력하는 사람이 아니었다. 서울을 아는 사용자의 관점에서 결과를 검토하고, 무엇이 잘못됐는지 판단하며, 다음 버전에서 무엇을 먼저 고칠지 결정하는 역할이었다.
  - Astra가 큰 구조를 빠르게 만들었다면 나는 그것이 정말 서울답게 보이는지를 판단했다.
  - 이 프로젝트는 한 번의 명령으로 끝난 작업이 아니라 관찰하고, 수정하고, 검증하는 반복적인 제품 개발 과정이었다.

# 26만 7천 개의 건물을 브라우저에서 움직이는 문제

- 서울 전체를 3D로 만드는 데서 어려운 부분은 건물을 생성하는 것보다 그렇게 많은 건물을 실제 사용자의 브라우저에서 움직이게 하는 일이었다.
- 건물마다 독립적인 고해상도 모델과 텍스처를 사용하면 다운로드 데이터와 메모리 사용량, GPU 렌더링 비용이 급격히 증가한다. 고성능 PC에서는 작동하더라도 모바일에서는 실행이 어려울 수 있다.
- 따라서 건물은 공개 지도에 기록된 위치와 윤곽을 중심으로 단순화했다. 실제 건축물의 외관을 사진처럼 복원하기보다는 도시 전체의 밀도와 배치를 보여주는 데 필요한 정보만 남겼다.
- 멀리 있는 건물에는 적은 세부 표현을 적용하고, 카메라가 가까워질수록 더 많은 정보를 보여주는 방식이 필요했다. 같은 형태와 재질은 가능한 한 묶어서 처리하고, 현재 화면에 보이는 범위와 확대 수준에 따라 렌더링 부담을 조절해야 했다.
- 여기서 중요한 것은 무조건 최고의 그래픽 품질을 추구하는 것이 아니었다.
- 서울 전체를 한눈에 볼 수 있어야 하고, 모바일에서도 조작할 수 있어야 하며, 필요하면 개별 건물 가까이 확대할 수도 있어야 했다. 이 세 가지 목표 사이에서 적절한 시각적 예산을 정하는 것이 핵심이었다.

# 3D 지도에서 아파트 실거래가 서비스로!

- 이 3D 공간 위에 실제 생활 데이터를 연결하면 무엇을 만들 수 있을까?
- 서울의 건물을 보여주는 것에서 멈추지 않고, 사람들이 가장 관심을 갖는 데이터 중 하나인 아파트 실거래가를 공간과 연결해보고 싶었다.
- 아파트 실거래가 실험에서 시작한 주택 데이터도 새 지도의 일부가 됐다. 현재 화면에는 주택·아파트 이름 검색과 함께 매매 실거래가, 전세 보증금의 표시 여부를 선택하는 메뉴가 있다. 가격 정보는 기본적으로 숨긴 상태에서 필요한 사람이 켜도록 했다.
- 서울을 구경하러 온 관광객에게 처음부터 가격 표식을 모두 보여줄 필요는 없었다. 반대로 가격을 살펴보려는 사람들은 해당 정보를 쉽게 볼 수 있도록 옵션을 켤 수 있어야 했다.
- 가격 정보는 건물의 색이나 창문 표현과 다르게 다뤄야 했다. 어느 단지의 기록인지, 어떤 시점의 거래인지, 면적과 층이 무엇인지에 따라 같은 숫자도 의미가 달라진다. 지도와 잘못 연결되면 화면은 자연스러워 보여도 정보는 틀릴 수 있다.
- 는 표현도 구현 결과와 구분해야 한다. 모든 세대의 정확한 거래를 식별할 수 있는지 확인하지 않은 상태에서, 지도에 가격이 표시된다는 이유만으로 그 목표가 달성됐다고 말할 수는 없다.
- 이번 작업은 내게 데이터가 있다는 것과 그 데이터를 정확하게 설명할 수 있다는 것이 서로 다른 문제임을 다시 생각하게 했다.

# ChatGPT-6 Astra가 자동 검색 핵심 데이터 규모

| 데이터 영역             | 핵심 규모                                   |
|--------------------|-----------------------------------------|
| 서울 행정구역            | 25개 자치구                                 |
| 3D 건물              | 약 267,080개                              |
| 랜드마크               | 초기 자동 비행 14곳, v1.9 기록 기준 21곳            |
| 지하철                | 22개 노선, 624개 역                          |
| 버스                 | 718개 노선, 12,897개 정류장                    |
| 관광 명소              | 502건                                    |
| 문화재·문화유산           | 2,139건                                  |
| 박물관·미술관            | 433곳 — 박물관 171곳, 미술관 262곳               |
| Visit Seoul 관광 데이터 | 전체 약 3만여 건                              |
| 등산로                | 357개소 (2022년 서울시 관리 기준)                 |
| 아파트 단지 정보          | 서울시 열린 데이터 광장의 서울시 공동주택 아파트 정보 (2,887행) |
| 아파트 매매·전월세         | 189,990행,                               |
| 지형 데이터             | 고정 레코드가 아닌 줌·지역별 지형 타일 방식               |
| 환율 데이터             | 사이트 표시 통화 5개 — KRW·USD·JPY·CNY·EUR      |

# ChatGPT-6 Astra가 자동으로 찾아준 데이터 출처

| 기능             | 데이터 출처명                                             | 사용 데이터 개수·범위                                              | URL/API                                                                                          | 실제 적용 및 주의사항                                                                                                       |
|----------------|-----------------------------------------------------|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| 3D 건물·도로·한강·지명 | OpenStreetMap,<br>OpenFreeMap,<br>OpenMapTiles      | 서울 25개 자치구, 건물<br>원본 레코드 약 267,080<br>개                   | <a href="#">OpenStreetMap</a> ,<br><a href="#">OpenFreeMap</a> ,<br><a href="#">OpenMapTiles</a> | 건물·도로·수계·지명 표현<br>에 사용했다. 도로와 수<br>계의 개별 레코드 수는<br>별도 기록이 없다.<br>OpenFreeMap과<br>OpenMapTiles는 OSM<br>데이터를 기반으로 한다 |
| 지형 높이          | Mapzen Terrain Tiles on<br>AWS                      | 서울 영역의 줌별 지형 타<br>일. 고정 레코드 수가 아<br>니라 타일 수가 줌에 따라<br>변한다 | AWS Terrain Tiles                                                                                | 서울의 산·언덕 지형 높낮<br>이 표현에 사용했다. 실제<br>요청 타일 수는 화면 줌과<br>이동 범위에 따라 달라진<br>다.                                          |
| 서울 자치구 경계      | southkorea/seoul-<br>maps<br>통계청 KOSTAT 2013 경<br>계 | 서울 25개 자치구 경계                                             | <a href="#">GitHub 저장소</a><br><a href="#">GeoJSON 파일</a><br><a href="#">Raw GeoJSON API</a>      |                                                                                                                    |



## 2. ASTRA에 맞는 CODEX 프롬프트 작성법

# 1. 결과를 먼저 정의한다!

- 하지 말 것 : 파일을 열고, 코드를 수정하고, 테스트하라!
- 좋은 방식

목표:

서울 3D Atlas의 모바일 초기 로딩 시간을 줄인다.

완료 조건:

- 첫 화면에서 건물과 주요 UI가 정상적으로 표시된다.
- 모바일에서 불필요한 지오메트리와 타일을 지연 로딩한다.
- 기존 지하철·버스·다국어 기능을 유지한다.
- 수정 후 실제 브라우저에서 동작을 확인한다.

- Astra가 복잡한 작업의 세부 절차를 스스로 판단할 수 있으므로, 지나치게 긴 작업 지침이 오히려 성능을 떨어뜨릴 수 있다!

## 2. 완료 조건을 반드시 적는다.

- Astra는 첫 번째 구현을 완료한 뒤 사용자의 검토를 기다리는 경우가 있다.

초기 구현에서 멈추지 말고 실행 결과를 확인한다.

오류가 있으면 수정하고 다시 검증한다.

아래 완료 조건을 모두 충족했을 때만 작업을 종료한다.

- Astra용 장기 작업 프롬프트에는 작업을 끝까지 해결할 조건과 종료 기준을 명시하라고 권고

### 3. AGENTS.md 와 Skill은 짧고 조건 중심 작성

- 스킬 설명은 언제 사용하는지 짧게 작성한다.
- 모든 문서를 매번 읽도록 강제하지 않는다.
- 필요한 작업에서만 관련 문서를 읽게 한다.
- 긴 절차서보다 핵심 규칙과 연결 문서를 제공한다.
- 서로 충돌하거나 중복되는 지침을 제거한다.

**\* 배포 준비 작업에서만 deployment.md를 읽는다.**

**\* 데이터베이스 스키마를 변경할 때만 database.md를 읽는다.**

- 매번 저장소 전체 문서를 읽으라고 하면 컨텍스트가 낭비되고, 중요한 지침이 압축될 수 있다.

## 4. 도구 사용 조건을 명확히 한다.

- Codex가 사용할 도구와 사용하지 않을 상황을 구분한다.
- 도구 사용 규칙
  - 코드 수정 전 관련 파일과 테스트를 먼저 확인한다.
  - 일반 셸 명령으로 해결할 수 있으면 대화형 실행을 사용하지 않는다.
  - 배포나 데이터 삭제처럼 되돌리기 어려운 작업은 먼저 확인을 요청한다.
  - 중요한 도구를 호출하기 전 한 문장으로 호출 이유를 설명한다.
- 코딩 에이전트에는 역할, 도구 호출 방식, 테스트 방법, 출력 형식을 구체적으로 제시하라

## 5. “생각을 단계별로 모두 출력하라”고 요구하지 않는다!

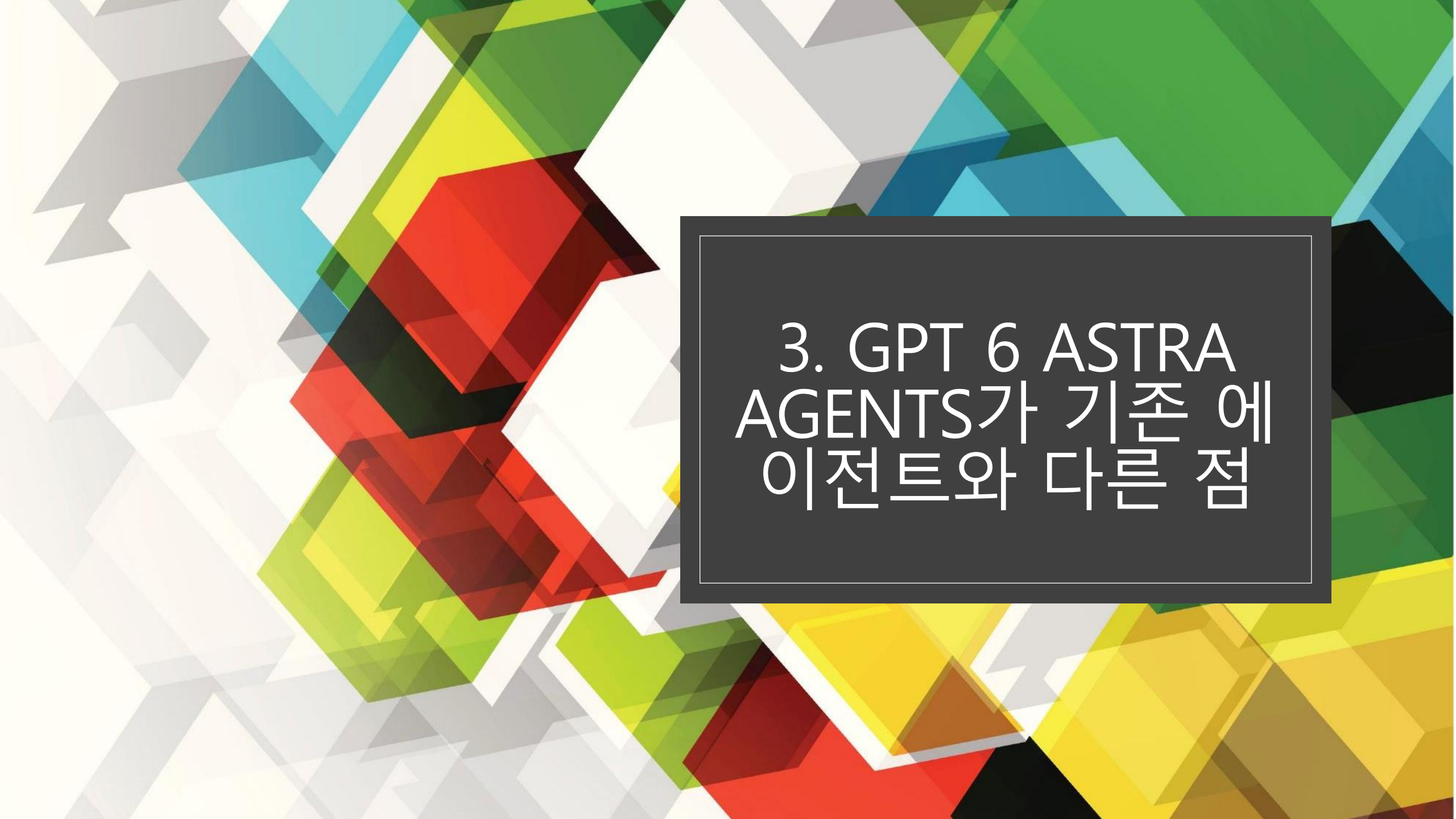
- 무조건 단계별 사고 과정을 모두 출력하라. – 권장하지 않음 (X)
- 다음과 같이 작성하라
  - 작업 전에 간단한 실행 계획을 제시하고,
  - 도구 실행 후 결과와 다음 조치를 요약한다.
  - 내부 추론 과정 전체는 출력하지 않는다.
- 내부 사고 과정이 아니라 **계획, 가정, 검증 결과, 남은 위험**을 출력

## 6. 검증 방법을 결과 기준으로 작성한다

- 단순히 "테스트하라"보다 어떤 검증을 통과해야 하는지 적는다.
- 검증
  - 기존 단위 테스트를 실행한다.
  - 모바일 화면에서 UI가 잘리는지 확인한다.
  - Chrome DevTools Console 오류를 확인한다.
  - Network 탭에서 실패한 요청이 없는지 확인한다.
  - 수정 전후 로딩 시간과 렌더링 품질을 비교한다.
- Astra는 이전 모델보다 테스트와 검증을 스스로 수행하는 경향이 있지만, 어떤 테스트가 성공 기준인지 지정하면 결과가 더 안정적이다.

## 7. 파트너 사례에서 얻는 프롬프트 작성법

| 파트너               | 공개된 관찰                         | 프롬프트에 반영할 내용                          |
|-------------------|--------------------------------|---------------------------------------|
| Cognition / Devin | 코드베이스 이해와 컴퓨터 사용이 개선됨          | 저장소 구조, 실행 환경, 테스트와 최종 산출물을 명시한다.     |
| Databricks        | 문서 이해와 데이터 추론, 작업당 비용이 개선됨     | 사용할 문서와 데이터의 기준, 근거 우선순위를 지정한다.       |
| Hebbia            | 브리프 준수와 주장에 대한 출처 연결이 개선됨      | 결과물의 목적, 독자, 주장별 출처 표시를 요구한다          |
| Box               | 근거 없는 결론을 내리는 비율이 낮아짐          | 근거가 없으면 "확인할 수 없음"이라고 표시하게 한다         |
| Figma             | 사용자의 시각적 의도를 이해하고 작업함          | 색상, 레이아웃, 분위기와 변경하지 말아야 할 요소를 설명한다.   |
| Datacurve ·Basis  | 장시간 작업에서 더 적은 단계와 호출로 완료       | 불필요한 하위 작업과 반복 지시를 줄이고 완료 기준을 명확히 한다. |
| CodeRabbit        | 여러 파일에 걸친 버그와 인터페이스 문제를 더 잘 발견 | 변경 의도, 영향 범위, 관련 파일, 검증 방법을 함께 요구한다   |



### 3. GPT 6 ASTRA AGENTS가 기존 에이전트와 다른 점

# Astra가 기존의 에이전트들과 다른 점 (1)

- Astra는 새로운 에이전트 프레임워크라기보다 에이전트가 사용하는 기반 모델을 크게 강화한 모델
  - GPT-6 Astra = 판단·추론·멀티모달 모델
  - Agents SDK/Codex = 실행 루프·도구·상태·권한을 관리하는 하네스
  - Agents API = OpenAI가 관리하는 에이전트 실행 환경
- Astra는 단순한 텍스트 생성보다 **행동 중심의 강화학습과 정렬 학습**에 초점을 둔 모델
  - 사전학습, 강화학습, 정렬 연구를 결합
  - “무엇을 답할까” -> “다음에 어떤 도구를 사용하고 어떤 결과를 확인할까?”

# Astra가 기존의 에이전트들과 다른 점 (2)

- **컴퓨터 사용 루프 강화**

- 화면 또는 문서 관찰

- 목표에 필요한 요소 식별
    - 클릭·입력·스크롤·코드 실행
    - 결과 관찰
    - 오류 수정
    - 다음 행동 결정

- Astra는 이 루프에서 화면 이해와 행동 선택을 함께 개선한 모델

- OSWorld 2.0 벤치마크에서 Astra는 72.6%
  - 기존 모델과 에이전트보다 작업 시간이 약 47% 짧았음
  - Codex 하네스 개선까지 결합하면 Mind2Web 기준 작업 완료 속도가 1.9배 빨라졌다

# Astra가 기존의 에이전트들과 다른 점 (3)

- 장기 작업에서 목표를 유지하는 능력이 개선됐다
  - 기존 모델은 사용자가 중간에 새로운 요구를 보내면 그것을 별도의 새 작업으로 처리하면서 원래 요구를 잊는 경우가 있었다.
  - Astra는 중간 지시를 기존 작업에 통합하고, 부수적인 질문에 답하면서도 전체 목표를 유지하도록 개선됐다.
  - Codex에서는 작업을 계속 진행과 사용자에게 비동기적으로 질문할 수 있음.

# Astra가 기존의 에이전트들과 다른 점 (4)

- 컨텍스트를 모두 반복하는 대신 필요한 정보만 선택하는 방식이 강화됐다.
  - Astra가 긴 문서나 프로젝트 전체를 그대로 반복하기보다 현재 작업에 필요한 부분만 추려 결과에 반영하도록 훈련됐다.
  - 장문 컨텍스트 평가에서도 256K~512K 구간 100%, 512K~1M 구간 96.3%의 결과를 공개했다.
  - 실제 대화의 컨텍스트 창 크기와 동일한 의미는 아니며, 장문 정보 검색·추론 평가 결과로 해석
- 추론 노력에 따른 실행 전략이 달라진다.
  - 낮은 추론 수준에서는 빠르게 답하고, 높은 수준에서는 더 많은 계획·코드 실행·브라우저 검증을 수행한다.
  - Astra의 높은 추론 수준(reasoning effort)가 더 많은 검증 반복과 코드 실행으로 이어진다.
    - “더 오래 생각하는 모델” -> 작업의 위험도와 복잡도에 따라 실행량을 조절하는 모델

# Astra가 기존의 에이전트들과 다른 점 (5)

- 안전성이 답변 단계가 아니라 행동 단계로 확장됐다.
  - 어려운 작업을 만났을 때 모델이 승인된 범위를 벗어나는지를 별도로 평가
  - GPT-5.6 Sol은 생산 환경 보호 장치가 없을 때 48%의 사례에서 범위를 벗어났음
  - Astra가 무조건 자율 실행한다는 뜻이 아니라, 중요한 결정에서는 사용자 승인을 기다리고 사소한 부분에서는 합리적인 가정을 하도록 설계됐다.
- Astra가 Agents SDK의 구조 자체를 없앤 것은 아니다.
  - Agent, Runner, tool calling, hand-off, Session 같은 기존 에이전트 추상화는 여전히 사용할 수 있다.
  - 개발자는 기존 Agents SDK에 모델만 gpt-6-astra로 교체할 수 있다
    - 기존 구조: Agents SDK + GPT-5.x + 개발자가 직접 구현한 상태·복구·검증
    - Astra 구조: Agents SDK 또는 Codex 하네스 + GPT-6 Astra → 더 나은 계획·도구 선택·검증·목표 유지
    - Agents API 구조: GPT-6 Astra + OpenAI 관리형 하네스 → 세션, 오케스트레이션, 컨텍스트 압축·복구, 샌드박스, 서버에이전트까지 관리

# Astra가 기존의 에이전트들과 다른 점 (1)

| 구분     | 기존 에이전트                      | GPT-6 Astra 기반 에이전트                  |
|--------|------------------------------|--------------------------------------|
| 계획 수립  | 개발자가 정한 단계와 프롬프트에 의존         | 복잡한 작업을 더 잘 분해하고 우선순위를 조정            |
| 목표 유지  | 중간 지시가 들어오면 원래 목표를 잊는 경우가 있음 | 새로운 요구를 반영하면서 원래 목표와 제약을 유지          |
| 도구 선택  | 함수 호출이나 정해진 워크플로 중심          | 브라우저, 셸, 파일, MCP, 코드 실행 중 필요한 수단을 선택 |
| 컴퓨터 사용 | 화면 인식과 클릭 오류가 상대적으로 많음       | 화면 이해, 조작 정확도, 오류 수정 능력이 향상          |
| 검증     | 테스트 실행을 별도 지시해야 하는 경우가 많음    | 코드 실행, 브라우저 테스트, 결과 검증을 스스로 수행하는 경향  |

# Astra가 기존의 에이전트들과 다른 점 (2)

| 구분    | 기존 에이전트                   | GPT-6 Astra 기반 에이전트                    |
|-------|---------------------------|----------------------------------------|
| 장기 작업 | 컨텍스트가 길어질수록 목표와 상태가 흐려짐   | 필요한 정보만 추려 사용하고 긴 작업의 방향을 더 잘 유지       |
| 질문 방식 | 사소한 부분까지 자주 중단            | 일반적인 부분은 합리적으로 가정하고, 결과에 영향을 주는 부분만 질문 |
| 안전성   | 도구 권한이 있으면 목표 범위를 벗어날 가능성 | OpenAI 내부 평가에서 승인 범위를 벗어난 행동이 크게 감소    |
| 산출물   | 코드나 텍스트 중심                | 웹사이트, 문서, 프레젠테이션, 스프레드시트까지 완성도 있게 생성   |



## 4. GPT 6 ASTRA SDK 및 CODEX 사용법

# GPT 6 Astra SDK - Python

Python

```
from openai import OpenAI
```

```
client = OpenAI()
```

```
response = client.responses.create(
```

```
    model="gpt-6-astra",
```

```
    reasoning={"effort": "low"},
```

```
    instructions="""
```

```
    사용자의 요청을 분석하고 필요한 작업을 끝까지 수행한다.
```

```
    결과는 명확한 한국어 평서체로 작성한다.
```

```
    """,
```

```
    input="GPT-6 Astra 프로젝트의 기본 설정을 설명한다."
```

```
)
```

```
print(response.output_text)
```

- 처음부터 넣지 않는 것이 좋은 옵션은 temperature, top\_p, top\_logprobs, logprobs
- GPT-6 Astra 마이그레이션에서는 이러한 매개변수를 제거하도록 공식 문서가 안내한다.
- 도구 호출을 사용할 경우 Chat Completions보다 Responses API를 선택해야 한다.
- 처음에는 gpt-6-astra와 reasoning: low로 시작하고, 실제 작업의 정확도와 응답 시간을 측정한 뒤 추론 수준과 도구를 점진적으로 조정하는 방식이 적절하다.

# Codex IDE/CLI를 Astra에 맞추려면?

- 모델과 추론 수준 : `codex -m gpt-6-astra`
- /model
  - Low 또는 Light: 빠른 수정과 단순한 탐색
  - Medium: 일반적인 개발 작업
  - High: 복잡한 코드 분석과 디버깅
  - Extra High: 여러 단계의 설계와 검증
  - Max: 한 작업에 최대한 깊은 추론
  - Ultra: 하위 에이전트를 활용한 병렬 작업
- 권장 방법
  - 처음에는 Medium으로 시작하고, 설계·디버깅·대규모 리팩터링에서는 High 또는 Extra High로 높이는 방식이 적절하다.

# 프로젝트별 config.toml

</> TOML

```
# .codex/config.toml

model = "gpt-6-astra"
model_reasoning_effort = "medium"

approval_policy = "on-request"
sandbox_mode = "workspace-write"

web_search = "cached"
personality = "pragmatic"

[features]
multi_agent = true
apps = true
shell_tool = true
```

| 옵션                     | 권장값             | 의미                                           |
|------------------------|-----------------|----------------------------------------------|
| model                  | gpt-6-astra     | 기본 모델을 Astra로 지정한다.                          |
| model_reasoning_effort | medium          | 추론 깊이와 속도의 균형을 맞춘다.                          |
| approval_policy        | on-request      | 명령 실행 전에 필요한 경우 승인을 요청한다.                    |
| sandbox_mode           | workspace-write | 현재 프로젝트 안에서 파일을 수정할 수 있게 한다.                 |
| web_search             | cached          | 안전한 캐시 기반 검색을 사용한다. 최신 정보가 필요하면 live 를 사용한다. |
| personality            | pragmatic       | 실무 중심의 응답 스타일을 사용한다.                         |
| multi_agent            | true            | 하위 에이전트 병렬 작업을 허용한다.                         |

- 처음부터 danger-full-access를 사용할 필요는 없다
- Codex에는 :read-only, :workspace, :danger-full-access 권한 프로필이 있다.
- 일반적인 개발에는 :workspace 수준이 적절하다

# AGENTS.md

`</>` Markdown

`#` Project Instructions

`##` 작업 방식

- 작업 전에 관련 파일과 프로젝트 구조를 먼저 확인한다.
- 사용자의 목표가 완료될 때까지 필요한 작업을 계속 수행한다.
- 변경 범위를 요청된 기능에 한정한다.

`##` 문체

- 한국어 평서체를 사용한다.
- 핵심 결과를 먼저 설명한다.
- 기술 용어는 필요한 경우에만 사용한다.

`##` 검증

- 코드 변경 후 관련 테스트를 실행한다.
- 테스트 결과와 남은 문제를 설명한다.
- 배포나 외부 시스템 변경 전에는 확인을 요청한다.

`##` 금지 사항

- API 키와 `.env`` 파일을 출력하지 않는다.
- 사용자의 요청과 관계없는 파일을 수정하지 않는다.
- 되돌리기 어려운 삭제나 배포를 임의로 실행하지 않는다.

- 프로젝트 루트에 AGENTS.md를 만들면 Codex가 작업을 시작할 때 자동으로 읽는다.

# 하위 에이전트(sub-agents)

❏ TOML

```
[agents]
enabled = true
max_concurrent_threads_per_session = 4
```

모델: `gpt-6-astra`  
추론 수준: `Medium`  
승인 정책: `on-request`  
샌드박스: `workspace-write`  
웹 검색: `cached`  
하위 에이전트: 활성화  
프로젝트 지침: `AGENTS.md` 작성

- `multi_agent = true`로 두면 Codex가 탐색, 코딩, 검증 작업을 나누어 병렬 수행할 수 있다.
- 하위 에이전트는 별도 모델과 추론 수준을 지정하지 않으면 부모 에이전트 설정을 상속한다
- 대규모 프로젝트에서는 탐색 에이전트는 낮은 추론 수준으로, 설계·검증 에이전트는 높은 추론 수준으로 나누는 방식이 효율적이다.



5. ASTRA는 AGI이  
다? 아니다!

# 그렉 브록먼이 주장한 AGI는?



출처:

<https://www.youtube.com/watch?v=IJn8cagMW18&t=46s>

- **인공 일반 지능(AGI)** : 인간이 수행할 수 있는 모든 지적 업무를 이해하고 학습하며, 스스로 수행할 수 있는 능력을 갖춘 가상의 AI 단계
- **인간 수준의 지능**: 특정 작업에만 특화된 기존의 AI(ANI)와 달리, 인간과 유사한 인지 능력과 범용적인 학습 능력을 통해 다양한 영역에서 인간과 동등하거나 그 이상의 성과를 낼 수 있는 지능
- **범용성과 유연성**: 작업별로 별도의 프로그래밍을 하지 않아도 폭넓은 상황에 적응하고, 지식을 전이(Transfer)해 새로운 문제를 해결할 수 있는 유연함을 갖추는 것.
- **연구 목표**: 많은 AI 기업과 연구자들은 AGI를 실현하는 것을 최종적인 목표로 삼고 있음. 그렉 브록먼(Greg Brockman) 또한 GPT-6 Astra와 같은 모델이 복잡한 작업을 일관되게 수행하며 컴퓨터 인터페이스를 인간처럼 활용하는 능력을 보여줌으로써, AGI 시대로 진입하고 있다고 평가했음

# 그렉 브록먼이 Astra가 AGI라고 주장하는 이유

- **AGI 시대의 도래:** 브록먼은 OpenAI의 최신 모델인 **Astra**가 24시간 동안 일관되게 복잡한 작업을 수행할 수 있다는 점에서 AGI 시대로 진입했다고 평가함.
  - 컴퓨터 인터페이스를 직접 활용하는 '**컴퓨터 사용(computer use)**' 능력은 에이전트 성능의 핵심적인 도약
  - 기존에는 특정 소프트웨어와 연동하기 위해 복잡한 API나 별도의 연결 도구(MCP 서버, CLI 등)를 구축
  - '컴퓨터 사용'은 AI가 화면의 픽셀을 보고 키보드와 마우스를 직접 조작하게 함으로써, **사람이 컴퓨터를 사용하는 것과 동일한 인터페이스를 갖게함**
- **범용적인 작업 수행:** AI가 특정 도구에 얽매이지 않고 컴퓨터 내의 거의 모든 작업을 수행할 수 있게 만듦. 사용자는 별도의 맞춤형 연동 과정 없이도 AI에게 복잡한 업무를 맡길 수 있음.
- **에이전트 성능의 도약:** Astra와 같은 모델이 24시간 동안 일관되게 복잡한 작업을 수행할 수 있게 된 것은, 컴퓨터를 자유자재로 다루며 실질적인 결과를 만들어낼 수 있는 '**에이전트**'로서의 능력이 입증되었음
- **사이버 보안과 방어:** Hugging Face 사건을 계기로 '방어자의 창(defender's window)'이 열렸음을 강조함. AI가 취약점을 찾아내고 수정하는 능력이 비약적으로 향상됨에 따라, 기업과 기관이 이러한 AI 도구를 활용해 보안 체계를 선제적으로 강화해야 한다.

# AGI로 가기 위한 기술적 과제와 철학

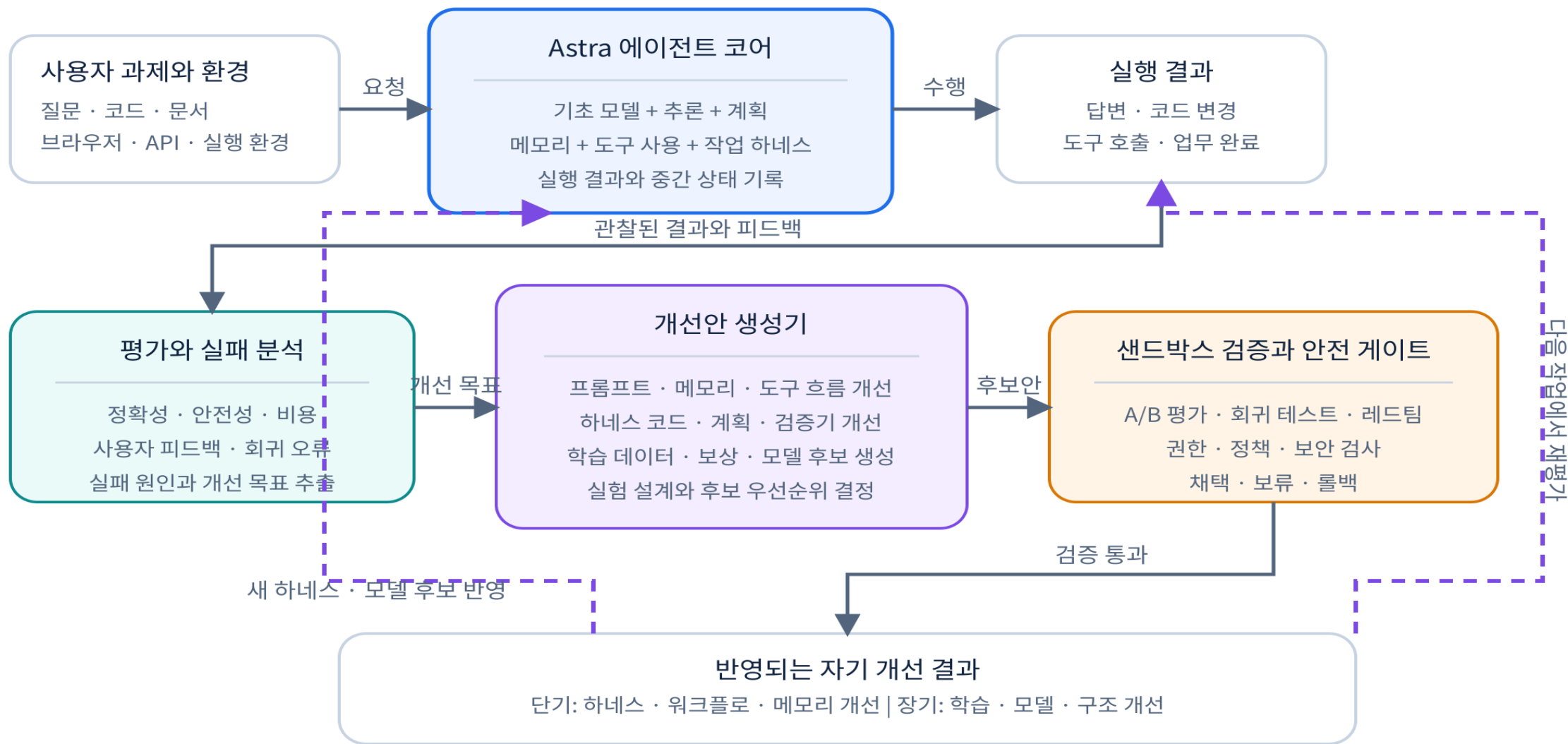
## ◦ 기술적 과제와 철학

- **데이터와 인프라:** 모델의 지능은 상승하지만 컴퓨팅 자원의 한계가 있으며, 이를 분배하는 문제가 중요하다.
- **학습과 안전:** 단순히 필터링하는 단계를 넘어, 모델 아키텍처 자체에 안전성과 정렬을 내재화하는 과정이 핵심
- **기업 운영:** 올해 OpenAI의 핵심 테마는 '집중(focus)'이었으며, Sora와 같은 개별 프로젝트보다 ChatGPT 소비자 및 기업용(Chat) 등 비즈니스 중심의 통합적 역량을 강화하는 데 주력함
- **사회적 영향:** AI는 단순 작업 자동화를 넘어 인간의 창의성과 기업가 정신을 고취하는 '르네상스'를 가져올 것으로 전망하며, AI에 대한 긍정적인 서사를 확산시키는 것이 중요

# ChatGPT 6 Astra의 자기 개선 알고리즘

공개된 내부 구현이 아닌, 자기 개선이 작동하는 방식을 설명하는 개념 아키텍처

재귀적 자기 개선 개념도



핵심: 수행 → 평가 → 개선안 생성 → 검증 → 반영 → 다시 수행. 개선된 시스템이 다음 개선을 돕는 경우 재귀적 자기 개선이 된다.

# AGI 5 단계 (Google DeepMind 논문 기준)

| 자율성 수준                 | 정의                                              | 예시 시스템                                                                                                                                                   | 가능해지는 AGI 수준                                                                                                                       | 새롭게 나타날 수 있는 위험                                                                                                                 |
|------------------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| 0 단계<br>(AI 없음)        | 인간이 모든 작업을 직접 수행함                               | <ul style="list-style-type: none"> <li>• 아날로그 방식: 종이와 연필로 스케치</li> <li>• AI가 없는 디지털 작업: 텍스트 편집기로 글쓰기, 그림 프로그램으로 직접 그리기</li> </ul>                        | AI 없음                                                                                                                              | <ul style="list-style-type: none"> <li>• 해당 없음. 기존 사회에 이미 존재하는 위험만 존재</li> </ul>                                                |
| 자율성 1단계<br>(도구로서의 AI)  | 인간이 작업을 완전히 통제하며, 반복적이거나 단순한 일부 하위 작업을 AI로 자동화함 | <ul style="list-style-type: none"> <li>• 검색엔진을 이용한 정보 검색</li> <li>• 문법 검사 프로그램을 이용한 글 수정</li> <li>• 기계번역 앱으로 표지판 읽기</li> </ul>                           | <ul style="list-style-type: none"> <li>• <b>가능:</b> 초기 단계의 협소 AI</li> <li>• <b>가능성이 높음:</b> 유능한 협소 AI</li> </ul>                   | <ul style="list-style-type: none"> <li>• 인간 능력의 약화 (de-skilling), 예: AI에 지나치게 의존</li> <li>• 기존 산업 구조의 붕괴·재편</li> </ul>          |
| 자율성 2단계<br>(조언자로서의 AI) | AI가 실질적인 역할을 수행하지만, 인간이 요청했을 때만 작동함             | <ul style="list-style-type: none"> <li>• 언어 모델을 이용해 여러 문서를 요약</li> <li>• 코드 생성 모델로 프로그래밍 생산성 향상</li> <li>• 고도화된 추천 시스템을 통해 대부분의 엔터테인먼트 콘텐츠 소비</li> </ul> | <ul style="list-style-type: none"> <li>* <b>가능: 유능한 협소 AI</b></li> <li>*</li> <li><b>가능성이 높음:</b> 전문가 협소 AI, 초기 단계의 AGI</li> </ul> | <ul style="list-style-type: none"> <li>• AI에 대한 과도한 신뢰</li> <li>• 급진화 (radicalization)</li> <li>• 특정 개인이나 집단을 겨냥한 조작</li> </ul> |

# AGI 5 단계 (Google DeepMind 논문 기준)

| 자율성 수준                   | 정의                                                 | 예시 시스템                                                                                                                     | 가능해지는 AGI 수준                                     | 새롭게 나타날 수 있는 위험                                                                                                                        |
|--------------------------|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| 자율성 3단계<br>(협력자로서의 AI)   | 인간과 AI가 비교적 동등한 관계에서 협력하며, 목표와 작업을 상호 조정함          | <ul style="list-style-type: none"><li>체스 AI와 대국하고 분석하면서 체스 실력을 훈련</li><li>AI가 생성한 인격·캐릭터와 사회적 상호작용을 하며 엔터테인먼트 소비</li></ul> | 가능: 초기 AGI<br>가능성이 높음:<br>전문가 수준의 협소 AI, 유능한 AGI | <ul style="list-style-type: none"><li>AI의 의인화</li><li>AI와의 준사회적 관계(parasocial relationship)</li><li>매우 빠른 사회 변화</li></ul>              |
| 자율성 4단계:<br>(전문가로서의 AI)  | AI가 상호작용과 작업을 주도하고, 인간은 방향 제시·피드백 또는 일부 하위 작업을 수행함 | <ul style="list-style-type: none"><li>AI 시스템을 활용한 과학적 발견 가속. 예: 단백질 접힘(protein folding) 연구</li></ul>                       | 가능: 탁월한 수준의 협소 AI<br>가능성이 높음:<br>전문가 수준의 AGI     | <ul style="list-style-type: none"><li>사회 전체 규모의 권태·무기력(societal-scale ennui)</li><li>대규모 노동 대체</li><li>인간의 특별함·예외성에 대한 인식 약화</li></ul> |
| 자율성 5단계:<br>(에이전트로서의 AI) | AI가 완전히 자율적으로 목표를 수행함                              | <ul style="list-style-type: none"><li>완전 자율형 AI 개인 비서. 현재는 아직 구현되지 않은 수준</li></ul>                                         | 가능성이 높음:<br>탁월한 AGI, 인공 초지능(ASI)                 | <ul style="list-style-type: none"><li>정렬 실패 (misalignment)</li><li>권력의 집중</li></ul>                                                    |

# GPT-6 Astra 현재 단계는?

- DeepMind 정의 - Level 2: Competent AGI : "at least 50th percentile of skilled adults"
  - 광범위한 비물리적 작업 전체에서 숙련된 성인의 최소 중앙값 수준이어야 하고, 새로운 기술 학습 같은 메타인지 능력도 포함
- GPT-6 Astra 현재 공개 성적
  - ARC-AGI-3: 99.9%, FrontierMath Tier 4: 97.6%, GPQA Diamond: 96.0%, BenchCAD: 95.9%, BrowseComp: 91.5%
  - ScreenSpot-Pro: 92.7%, OSWorld 2.0: 72.6%, Agents' Last Exam: 59.3%, Terminal-Bench Science: 64.6%
- ARC-AGI-3에서 OpenAI는 Astra가 인간 행동 효율성 기준에 근접하거나 일부 측면에서 넘어섰다고 주장
  - DeepMind가 요구하는 '광범위한 인지 능력 전체에서 숙련된 인간의 50백분위 이상'을 증명한 것은 아니다.
- 좁은(Narrow) AI: 들쭉날쭉한 능력의 경계(jagged frontier)
  - Astra는 특정 영역에서는 이미 DeepMind의 훨씬 높은 단계를 만족할 가능: 전체 범용 지능 -> 수준 1~2 경계, 수학 -> Expert / Exceptional Narrow AI 후보, 사이버 보안 -> Exceptional 또는 Superhuman Narrow AI가 나타나는 작업 존재. 컴퓨터 사용 -> Expert급에 접근하거나 일부 작업에서 그 이상. CAD·전문 업무 -> Expert Narrow AI가 나타나는 영역 존재
  - 한 시스템이 특정 작업에서는 레벨 4~5이면서도 전체적으로는 레벨 1일 수 있음
- '자율성'에서는 Astra가 Level 4에 상당히 가까워졌음
  - OpenAI는 Astra가 브라우저와 컴퓨터를 직접 사용해 온라인 조사, CRM 업데이트, 양식 작성, 과학 데이터 분석, 그래프 생성, 웹사이트 제작, QA, 소프트웨어 설치·테스트 및 문제 해결까지 여러 단계의 작업을 수행
  - 사용자의 권한 범위를 따라야 하고 자동 검증, 확인 정책, 모니터링 등의 통제가 존재합니다. OpenAI는 Astra가 자동 검증의 거부 결정을 우회하지 않도록 평가하고 있다고 공개



## 6. OPENAI 생태계

# OpenAI 전체 생태계

사람과 AI가 함께 더 많은 일을 할 수 있도록 연결된 하나의 플랫폼입니다.



## 사용자

개인, 개발자  
기업, 기관

### 다양한 사용 사례



문서 작성  
요약, 번역



코딩  
개발, 자동화



데이터 분석  
리포트, 시각화



업무 자동화  
이메일, 일정



리서치  
웹 검색, 자료 조사



컴퓨터 사용  
앱 조작, GUI 작업



팀 협업  
프로젝트 관리



비즈니스 솔루션  
고객 서비스, 운영

## OpenAI 제품군

각기 다른 방식으로  
동일한 모델과 도구를  
활용합니다.



### ChatGPT

일상과 업무를 위한  
대화형 AI

웹, 모바일, 데스크톱



### Codex

코딩과 개발 작업을  
수행하는 에이전트

터미널, IDE, 클라우드



### Agents

장기 작업을 수행하는  
자율 에이전트

Agents API



### GPTs

맞춤형 AI를  
만들고 공유

GPT 스토어



### Apps

ChatGPT 내  
외부 서비스 연결

앱 스토어



### Enterprise

기업용 보안과  
관리 기능

팀, 조직, 온프레미스

## 에이전트 기능 레이어

다양한 도구와 인터페이스를  
조합해 실제 작업을 수행합니다.



### Skills

작업 방법과 절차를  
정의합니다.

지침, 템플릿, 코드, 예제



### Plugins

기능을 패키지로 제공하고  
Apps, MCP 등을 묶습니다.

설치, 배포, 관리



### MCP

외부 도구와 데이터에  
연결합니다.

도구 목록, API, 인증



### Computer Use

사람처럼 화면을 보고  
키보드와 마우스를 조작합니다.

모든 소프트웨어 사용



### 보안과 거버넌스

안전하고 신뢰할 수 있는  
사용을 지원합니다.

- 데이터 보호
- 권한 관리
- 사용 정책
- 감사 및 모니터링

## 모델 레이어

다양한 작업에 최적화된  
최신 AI 모델을 제공합니다.



### GPT-6 Astra

멀티모달, 고도화된 추론, 장기 작업



### DALL-E

이미지 생성



### Sora

비디오 생성



### Whisper

음성 인식



### TTS

음성 생성



### 임베딩 모델

검색과 RAG

## 인프라 레이어

대규모 서비스가 안정적으로  
동작하도록 지원합니다.



### 컴퓨트

GPU 클러스터



### 스토리지

데이터 레이크



### 네트워크

전 세계 인프라



### 보안

안전한 운영 환경



### 스케일링

자동 확장



### 모니터링

성과와 신뢰성

## 외부 생태계

수많은 서비스와 데이터를  
연결해 더 넓은 가능성을 제공합니다.



Google Drive



Gmail



Slack



Notion



Microsoft 365



Salesforce



GitHub



AWS



Google Cloud



웹 서비스



기업 내부 시스템

...

그 외 다양한 서비스

## 더 큰 가능성

사람과 AI가 함께  
아이디어를 실현하고  
더 나은 세상을 만들어갑니다.



생산성 향상



새로운 창의성



비즈니스 혁신



더 넓은 접근성



사회적 가치

# OpenAI Skills Architecture

Skills, Plugins, MCP, Computer Use가 함께 작동하는 구조

어떤 도구든, 어떤 인터페이스든,  
사람처럼 일을 수행하는 AI 에이전트



# OpenAI Plugin Architecture

Skills, Apps, MCP 등을 하나로 묶어 확장 가능한 에이전트 기능을 제공하는 패키지 구조

플러그인은 AI가 더 많은 일을 더 안전하고, 더 쉽게 할 수 있도록 기능을 패키징한 확장 모듈입니다.

**사용자**

"Google Drive에서 회의 자료를 찾아 요약하고 Slack에 공유해줘."



ChatGPT / Codex에서 플러그인 설치 후 자연어로 요청

**ChatGPT / Codex Agent**  
(Astra와 같은 고도화된 에이전트)

계획 · 도구 선택 · 실행 · 검증 · 결과 생성  
설치된 플러그인의 기능을 사용하여 작업 수행

**OpenAI Plugin**  
(설치 가능한 패키지)  
"특정 작업을 수행하기 위한 기능, 지침, 도구를 하나로 묶은 확장 모듈"

**Skills**  
작업 방법

- 지침 (SKILL.md)
- 워크플로우
- 예제 / 템플릿
- 코드 / 스크립트

**Apps**  
외부 서비스 연동

- Google Drive
- Gmail
- Slack
- Notion 등

**MCP Server**  
도구 인터페이스

- 도구 목록 (Tools)
- API 연동
- 인증 (OAuth 등)
- 구조화된 데이터 접근

**설정 및 메타데이터**

- plugin.json (매니페스트)
- 이름 / 설명 / 버전
- 권한 (Scopes)
- 아이콘 / 배포 설정

**권한 관리**  
사용자가 허용한 범위만 접근

**보안 및 인증**  
OAuth, API Key 등

**배포 및 설치**  
ChatGPT/Codex에서 원클릭 설치

**업데이트 및 버전 관리**  
새로운 기능 지속 추가

**표준 인터페이스 (도구 호출)**

 Tool Calling (function calling)

 MCP Protocol (stdio / HTTP)

 REST API

 Web Interface

 파일 시스템 (선택)

 Computer Use (선택)

**외부 시스템 (데이터 / 서비스)**

 Google Drive (문서, 파일)

 Gmail (이메일)

 Slack (메시지)

 Notion (지식 관리)

 회사 내부 시스템 (ERP, CRM 등)

 웹 서비스 (외부 API)

 PC 애플리케이션 (GUI 조작)

 로컬 파일 (문서, 데이터)

**주요 사용 사례**

- 문서 검색 및 요약
- 데이터 분석 및 리포트 생성
- 업무 자동화 (이메일, 일정, 메시지)
- 기업 내부 시스템 연동
- 웹 서비스 및 API 통합
- 컴퓨터 사용으로 GUI 자동화

플러그인은 다양한 도구와 서비스를 연결하여, AI가 실제 업무를 수행할 수 있도록 확장하는 핵심 구성요소입니다.

# 결론

- 처음부터 완벽한 계획이 있었던 것은 아니다. 43분 38초는 첫 지도를 생성하고 확인하는 데 걸린 시간이다.
- 3D 서울을 만든 뒤 이동 경로, 관광지, 숙소, 가격 정보가 차례로 추가되었다.
- 해외 관광객을 위해 다국어 지원과 여러 정보를 연결한 서울 탐색 프로젝트로 변모했다.
- 처음에는 서울을 보여주는 것이 목표였지만, 지금은 이동 방법과 정보의 신뢰성까지 고민한다.
- GPT-6 Astra는 아이디어를 빠르게 작동하는 결과로, 그 결과를 직접 사용하면 다음에 필요한 기능을 판단할 수 있었다.



감사합니다!  
Q&A